

리눅스 CLI 도구 정리

bat · sd · yq · ffmpeg · pdftk · qpdf ·
ncdu · dust

실전 예제와 옵션으로 보는 파일 뷰어·치환·데이터·미디어·PDF·디스크 분석 도구

1

bat

cat의 업그레이드 버전 — 문법 하이라이팅을 지원하는 파일 뷰어

bat

bat — 개요 및 기본 문법

bat은 문법 하이라이팅, 줄 번호, git diff 표시를 지원하는 cat의 대체 도구입니다.

기본 문법

```
bat [옵션] 파일명
```

기본 사용 예시

```
# 기본 사용 (문법 하이라이팅 자동 적용)
bat main.py

# 여러 파일을 한 번에 보기
bat *.md

# 페이지저 없이 출력 (스크립트에서 활용할 때)
bat --paging=never config.yaml
```

bat — 자주 쓰는 옵션

옵션	설명
--line-range N:M	특정 줄 범위만 표시
--paging=never	페이저 없이 바로 출력
--diff	git diff 스타일로 변경 부분 표시
--theme <이름>	색상 테마 지정
--list-themes	사용 가능한 테마 목록 확인
-p / --plain	줄 번호·꾸밈 없이 출력

bat — 실전 활용 예제

1) 특정 줄 범위만 보기

```
bat --line-range 10:30 script.js
```

2) git diff 스타일로 변경 비교

```
bat --diff old.txt new.txt
```

3) 테마 변경해서 보기

```
bat --theme="Monokai Extended" app.js
```

4) grep 결과를 하이라이팅해서 보기

```
grep -l "TODO" *.py | xargs bat
```

2

sd

sed보다 직관적인 문법의 문자열 치환 도구

sd

sd — 개요 및 기본 문법

sd는 정규식 이스케이프가 단순하고 가독성이 좋은 sed 대체 도구입니다.

기본 문법

```
sd [옵션] "찾을패턴" "바꿀문자열" 파일명
```

기본 사용 예시

```
# 파일 내 문자열 치환 (기본은 stdout 출력)
```

```
sd 'foo' 'bar' file.txt
```

```
# 파일 자체를 수정 (in-place)
```

```
sd -i 'localhost:8080' 'api.example.com' config.json
```

sd

sd — 자주 쓰는 옵션

옵션	설명
-i / --in-place	파일을 직접 수정 (원본 덮어쓰기)
-s / --string-mode	정규식이 아닌 순수 문자열로 치환
-p / --preview	변경될 내용을 diff로 미리보기
-f / --flags	정규식 플래그 지정 (예: 대소문자 무시 i)

stdin과 파이프로 연결해 다른 명령어 출력에도 바로 적용할 수 있습니다.

sd

sd — 실전 활용 예제

1) 정규식 사용 (날짜 형식 변환)

```
sd '(\d{4})-(\d{2})-(\d{2})' '$3/$2/$1' dates.txt
```

2) 순수 문자열 치환 (특수문자 이스케이프 불필요)

```
sd -s 'old_string' 'new_string' *.py
```

3) stdin과 파이프로 연결

```
echo "hello world" | sd 'world' 'claude'
```

4) 여러 파일 일괄 치환 (find와 조합)

```
find . -name "*.js" | xargs sd 'var' 'const'
```

3

yq

YAML/JSON/XML 처리 도구 — jq의 YAML 버전

yq

yq — 개요 및 기본 문법

yq는 YAML, JSON, XML을 조회하고 수정할 수 있는 커맨드라인 처리 도구입니다.

기본 문법

```
yq [옵션] '쿼리' 파일명
```

기본 사용 예시

```
# 특정 값 읽기
yq '.spec.replicas' deployment.yaml

# 값 수정 (파일에 바로 반영)
yq -i '.spec.replicas = 5' deployment.yaml
```

yq — 자주 쓰는 옵션 & 문법

옵션 / 문법	설명
-i / --inplace	결과를 파일에 바로 반영
-o=json	출력 포맷을 JSON으로 변경
-p=json	입력 포맷을 JSON으로 지정
select(...)	조건에 맞는 문서·요소만 필터링
+=	배열 요소 추가
document_index	멀티 문서 YAML에서 특정 문서 선택
strenv(VAR)	환경변수를 YAML 값으로 주입

yq — 실전 활용 예제

1) 배열 요소 추가

```
yq -i '.spec.containers += [{"name": "sidecar"}]' pod.yaml
```

2) YAML → JSON 변환

```
yq -o=json '.' config.yaml > config.json
```

3) docker-compose에서 조건 필터링

```
yq '.services[] | select(.image == "nginx")' docker-compose.yaml
```

4) 환경변수를 YAML에 주입

```
yq -i '.env.API_KEY = stenv(API_KEY)' config.yaml
```

4

ffmpeg

오디오·비디오 처리의 표준 멀티미디어 도구

ffmpeg

ffmpeg — 개요 및 기본 문법

ffmpeg는 오디오·비디오의 변환, 편집, 인코딩을 위한 사실상 표준 도구입니다.

기본 문법

```
ffmpeg -i 입력파일 [옵션] 출력파일
```

기본 사용 예시

```
# 포맷 변환 (mov → mp4)
```

```
ffmpeg -i input.mov output.mp4
```

```
# 해상도 변경
```

```
ffmpeg -i input.mp4 -vf scale=1280:720 output_720p.mp4
```

ffmpeg — 자주 쓰는 옵션

옵션	설명
-i	입력 파일 지정
-ss / -t	시작 시간 / 길이 지정 (구간 자르기)
-vf	비디오 필터 적용 (스케일, 자막 등)
-c:v / -c:a	비디오·오디오 코덱 지정
-crf	화질 지정 (낮을수록 고화질, 압축률 ↓)
-vn	비디오 스트림 제외 (오디오만 추출)
-r	프레임레이트 지정

ffmpeg — 실전 활용 예제

1) 구간만 잘라내기 (1분30초부터 20초간)

```
ffmpeg -i input.mp4 -ss 00:01:30 -t 20 -c copy clip.mp4
```

2) GIF로 변환 (용량 최적화)

```
ffmpeg -i input.mp4 -vf "fps=10,scale=480:-1" output.gif
```

3) 오디오만 추출

```
ffmpeg -i video.mp4 -vn -acodec libmp3lame -q:a 2 audio.mp3
```

4) 영상 압축 (품질 유지하며 용량 절감)

```
ffmpeg -i input.mp4 -vcodec libx264 -crf 23 compressed.mp4
```

5

pdftk

PDF 병합·분할·암호화를 위한 조작 도구

pdftk — 개요 및 기본 문법

pdftk는 PDF 병합, 분할, 회전, 암호화 등 다양한 조작을 지원하는 도구입니다.

기본 문법

```
pdftk 입력.pdf 작업 output 출력.pdf
```

기본 사용 예시

```
# 여러 PDF 병합
pdftk file1.pdf file2.pdf cat output merged.pdf

# 특정 페이지만 추출 (1~5페이지)
pdftk input.pdf cat 1-5 output extracted.pdf
```

pdftk — 자주 쓰는 명령

명령 / 옵션	설명
cat	페이지를 이어붙여 결합·추출
1-endodd / 1-endeven	홀수 / 짝수 페이지만 선택
owner_pw / user_pw	편집 암호 / 열람 암호 설정
input_pw	암호 해제 시 기존 암호 입력
dump_data	메타데이터(제목, 저자 등) 확인
shuffle	두 PDF를 페이지 단위로 번갈아 병합
fill_form	FDF 데이터로 폼 필드 채우기

pdftk — 실전 활용 예제

1) 홀수 페이지만 추출

```
pdftk input.pdf cat 1-endodd output odd_pages.pdf
```

2) 암호 설정 (열람 암호 + 편집 방지)

```
pdftk input.pdf output secured.pdf owner_pw foopass user_pw readpass
```

3) 앞/뒷면 스캔본을 페이지 단위로 병합

```
pdftk A=odd.pdf B=even.pdf shuffle A B output interleaved.pdf
```

4) 폼 필드 채우기

```
pdftk form.pdf fill_form data.fdf output filled.pdf
```

6

qpdf

PDF 구조 최적화·저수준 변환 전문 도구

qpdf

qpdf — 개요 및 기본 문법

qpdf는 PDF 구조 최적화와 저수준 변환에 특화된, pdftk보다 세밀한 제어가 가능한 도구입니다.

기본 문법

```
qpdf [옵션] 입력.pdf 출력.pdf
```

기본 사용 예시

```
# PDF 선형화 (웹에서 빠른 로딩을 위한 최적화)
```

```
qpdf --linearize input.pdf output.pdf
```

```
# 암호 제거 (읽기 암호를 알고 있을 때)
```

```
qpdf --decrypt --password=secret input.pdf output.pdf
```

qpdf — 자주 쓰는 옵션

옵션	설명
--linearize	웹 최적화를 위한 선형화
--encrypt / --decrypt	암호 설정 / 해제
--pages	페이지 분할·병합·재구성
--rotate=각도:페이지	특정 페이지 회전
--check	PDF 구조 유효성 검사
--qdf	사람이 읽기 쉬운 QDF 모드 (복구용)
--object-streams=disable	손상된 PDF 복구 시 활용

qpdf — 실전 활용 예제

1) 페이지 분할

```
qpdf input.pdf --pages input.pdf 1-3 -- part1.pdf
```

2) 여러 PDF 병합

```
qpdf --empty --pages file1.pdf file2.pdf -- merged.pdf
```

3) 손상된 PDF 복구 시도

```
qpdf --qpdf --object-streams=disable broken.pdf repaired.pdf
```

qpdf --check로 구조를 먼저 점검하면 복구가 필요한지 빠르게 판단할 수 있습니다.

비교

pdftk vs qpdf — 비교

항목	pdftk	qpdf
주요 용도	병합·분할·암호화 등 범용 조작	구조 최적화·저수준 변환
문법	상대적으로 직관적	옵션이 세분화되어 정교함
PDF 복구	제한적	--qdf 등으로 손상 복구에 강함
웹 최적화(선형화)	미지원	지원 (--linearize)
폼 필드 채우기	지원 (fill_form)	미지원

7

ncdu

디스크 사용량을 대화형으로 탐색하는 TUI 도구

ncdu — 개요 및 기본 사용

ncdu는 디스크 사용량을 트리 형태로 탐색하며 큰 파일·디렉토리를 빠르게 찾는 대화형 도구입니다.

기본 사용

```
# 현재 디렉토리 분석 시작
ncdu

# 특정 경로 분석
ncdu /home/user

# 분석 결과를 파일로 저장 후 재열람 (재스캔 없이)
ncdu -o scan_result.json /var/log
ncdu -f scan_result.json
```

ncdu — 조작키 & 옵션

조작키 / 옵션	설명
d	선택한 파일·디렉터리 삭제
r	재스캔
n / s / c	이름순 / 크기순 / 항목수순 정렬
g	그래프 표시 형식 전환 (% / 막대그래프)
--exclude <패턴>	특정 디렉터리 제외하고 스캔
-o <파일>	스캔 결과를 파일로 저장

d 키로 삭제한 파일은 복구할 수 없으니, 삭제 전 항목을 다시 한번 확인하세요.

8

dust

du 대체 도구 — 디렉터리 사용량을 트리로 시각화

dust

dust — 개요 및 기본 문법

dust는 du의 대체 도구로, 디렉터리 사용량을 보기 쉬운 트리 형태로 즉시 출력합니다.

기본 문법

```
dust [옵션] [경로]
```

기본 사용 예시

```
# 현재 디렉토리 사용량을 트리로 출력
```

```
dust
```

```
# 특정 경로 분석
```

```
dust /var/log
```

dust

dust — 자주 쓰는 옵션

옵션	설명
-n <개수>	상위 N개 항목만 표시
-d <깊이>	트리 표시 깊이 제한
-A	숨김 파일 포함
-X <패턴>	특정 패턴 제외 (여러 번 지정 가능)
-f	파일까지 모두 펼쳐서 보기
-r	역순 정렬 (작은 것부터)

dust

dust — 실전 활용 예제

1) 상위 10개 항목만 표시

```
dust -n 10
```

2) node_modules, .git 제외하고 분석

```
dust -X node_modules -X .git
```

3) 파일까지 모두 펼쳐서 보기

```
dust -f
```

ncdu vs dust — 비교

항목	ncdu	dust
사용 방식	대화형 탐색 (TUI)	실행 즉시 결과 출력
삭제 기능	지원 (대화형 d 키)	미지원 (조회 전용)
결과 형태	트리 + 그래프(%)	트리 형태 요약
스크립트 활용	제한적 (-o/-f로 결과 저장)	출력을 파이프로 쉽게 활용
추천 상황	정리하며 직접 삭제할 때	빠르게 용량만 파악할 때

정리

실무 추천 워크플로우

1단계: dust로 큰 디렉터리 빠르게 파악

```
dust -n 10
```

2단계: ncdu로 상세 탐색하며 정리

```
ncdu .
```

3단계: yq/sd로 설정 파일 일괄 수정 + bat으로 결과 확인

```
yq -i '.image = "myapp:v2"' deploy.yaml && bat deploy.yaml
```

삭제·치환·암호화 옵션은 되돌릴 수 없으니, 실행 전 미리보기나 `-dryrun/-p` 옵션으로 반드시 먼저 확인하세요.